CMPEH 472: Lab 13,14,15: Raspberry Pi Pico W - PICO Project

Student Names: Mouli Grandhi and Michael Zimmerman

Lab Instructor: Terry Tirko

Date: Dec. 12th, 2025

Project Description: Our goal is to make a mobile robot controlled by an on-board Raspberry PI PICO W. The Robot is to be controlled by an infrared remote control with the receiver on-board the robot. The remote will send 5 different signals for left, right, forward, reverse, and stop. It has working turn signals and hazard lights, as well as a ultrasonic sensor that prevents the robot colliding with anything in front of it.

Lab 13 accomplishments: For this first lab, we focused on getting the robot built and getting the wheels to spin. We set up functions for forward, reverse, left turn, and right turn and tested it. We also began working on the PWM speed control but did not complete it.

Lab 14 accomplishments: This lab we tried to get the IR remote and receive to communicate and give us inputs we could use to drive the robot. We ended up trying a few different methods, eventually finding a driver that works well and got the all directional controls to work. We also tested the PWM motor control and were disappointed as the lower 70%-80% on the PWM range was not usable as it cannot consistently turn the motors.

Lab 15 accomplishments: We wrapped up the project by adding LEDs for turn signals and hazard lights, as well as a ultrasonic sensor to prevent the robot running into anything. The IR remote works well in range and the sensor stops the robot before it hits anything. The PWM motor speed was dropped as their wasn't enough adjustment to be worth adding the speed control. We instead used the PWM to set the motor speeds nearly equal so that the robot can drive straight. The project was a success and met the final project specifications.

Hardware:

- (1) Raspberry PI PICO W
- (1) Raspberry PI
- (2) DC motors
- (1) Motor drive module
- (1) Infrared remote
- (1) Infrared receiver
- (1) Switch
- (1) small breadboard for on-board circuit
- (2) Red LED
- (2) Yellow LED
- (2) 330 olm resistors
- Ultrasonic Sensor
- (1) AA battery pack
- Robotic equipment from the Raspberry PI Pico Advanced Kit

Lab 13:

Video: https://photos.app.goo.gl/qGZzD7LjBvdNRxRS7

Code:

```
#**************************************************
#Lab 13,14,15 - PICO Project
#Author: Mouli Grandhi and Michael Zimmerman
#Discription: Remote Control robot
#**************************************************

from machine import Pin, PWM ; imports
from time import sleep

LeftForward = 10 ; define GPIO pins
LeftBackward = 11
```

```
RightBackward = 12
RightForward = 13

lf = Pin(LeftForward, Pin.OUT) ; Initialize the pin-outs
lb = Pin(LeftBackward, Pin.OUT)
rf = Pin(RightForward,Pin.OUT)
rb = Pin(RightBackward, Pin.OUT)

def forward(): ; drive forward function
    lf.on()
    rf.on()
    lb.off()
    rb.off()

def backward(): ; drive backward function
    lb.on()
    rb.on()
    lf.off()
    rf.off()

def TurnRight(): ; turn right function
    lf.on()
    rb.on()
    lb.off()
    rf.off()

def TurnLeft(): ; turn left function
    lb.on()
    rf.on()
    lf.off()
```

rb.off()

while True: ; test loop

forward()

sleep(2)

backward()

sleep(2)

TurnLeft()

sleep(2)

TurnRight()

sleep(2)

Conclusion: Next lab we plan to complete the PWM control of the motors, as well as get the remote and receiver communicating and able to input values to start different functions to control the robot. We also talked about possibly installing a proximity sensor that will prevent the robot from driving into anything. We will do this if time permits.

Lab 14:

Video: https://photos.app.goo.gl/TiixQqNCrBcWdyLN9

Link to driver code: https://github.com/peterhinch/micropython_ir/blob/master/ir_rx/nec.py

Code:

```
#*****************************************************
#Lab 13,14,15 - PICO Project
#Author: Mouli Grandhi and Michael Zimmerman
#Discription: Remote Control robot
#*****************************************************

# Import Libraries
from machine import Pin, PWM
from ir_rx.nec import NEC_8
import utime

#-----------------------------------------------
# Define Pins
IRPinNum = 20
LeftForward = 10
LeftBackward = 11
```

```
RightBackward = 12
RightForward = 13

#--------------------------------------------------


#--------------------------------------------------
# Instantiating Pin modes
ir_pin = Pin(IRPinNum, Pin.IN, Pin.PULL_UP)
lf = PWM(Pin(LeftForward, Pin.OUT))
lf.freq(1000)
lb = PWM(Pin(LeftBackward, Pin.OUT))
lb.freq(1000)
rf = PWM(Pin(RightForward, Pin.OUT))
rf.freq(1000)
rb = PWM(Pin(RightBackward, Pin.OUT))
rb.freq(1000)
#--------------------------------------------------


#--------------------------------------------------
# Define Movement Functions

# Forward Movement
def forward():
    lf.duty_u16(64500)
    rf.duty_u16(65536)
    lb.duty_u16(0)
    rb.duty_u16(0)

# Reverse Movement
def backward():
    lb.duty_u16(645 00)
    rb.duty_u16(65536)
    lf.duty_u16(0)
    rf.duty_u16(0)

# Turn Right
def TurnRight():
    lf.duty_u16(53000)
    rb.duty_u16(0)
    lb.duty_u16(0)
    rf.duty_u16(0)

# Turn Left
def TurnLeft():
    lb.duty_u16(0)
    rf.duty_u16(53000)
    lf.duty_u16(0)
    rb.duty_u16(0)

# Stop
def stop():
    lf.duty_u16(0)
```

```
    rb.duty_u16(0)
    lb.duty_u16(0)
    rf.duty_u16(0)
#------------------------------------------------


#------------------------------------------------
# Maps inputs from IR remote code to movement function
def controller(code):
    if code == 0x18:
        forward()
    elif code == 0x52:
        backward()
    elif code == 0x08:
        TurnLeft()
    elif code == 0x5A:
        TurnRight()
    elif code == 0x1C:
        stop()
#------------------------------------------------


#------------------------------------------------
# Looks for button press (background process)

def callback(data, addr, ctrl):
    controller(data)

ir = NEC_8(ir_pin,callback)
#------------------------------------------------


#------------------------------------------------

# Makes Program run forever
try:
    while True:
        utime.sleep(.5)
except KeyboardInterrupt:
    stop()
#------------------------------------------------
```

Conclusion: The robot can be driven with the remote, it works well as long as it is in range and the remote is pointed at the receiver. We were disappointed with the PWM control as about 80% of the PWM range was unusable, so it is probably not worth including a speed control. We instead plan to use the PWM to balance the rpms of the wheels so it can drive straighter and turn a little slower. The final week we plan to add an additional feature as the PWM did not accomplish what we had hoped. We plan to add some kind of lights or a maybe a prox sensor, something along those lines to have another feature.

Lab 15:

Video: https://photos.app.goo.gl/mTYEEao4oGxGP4iv9

Code:

```
#*****************************************************
#Lab 13,14,15 - PICO Project
#Author: Mouli Grandhi and Michael Zimmerman
#Discription: Remote Control robot
#*****************************************************

from machine import Pin, PWM
from ir_rx.nec import NEC_8
from hcsr04 import HCSR04
import utime


dir = 0
distance = 100
forceStop = False
IRPinNum = 20
LeftForward = 10
LeftBackward = 11
RightBackward = 12
RightForward = 13


#-------------------------------------------------
#Defines all the pins
ir_pin = Pin(IRPinNum, Pin.IN, Pin.PULL_UP)
lf = PWM(Pin(LeftForward, Pin.OUT))
lf.freq(1000)
lb = PWM(Pin(LeftBackward, Pin.OUT))
lb.freq(1000)
rf = PWM(Pin(RightForward, Pin.OUT))
rf.freq(1000)
rb = PWM(Pin(RightBackward, Pin.OUT))
rb.freq(1000)
leftTurn = Pin(0,Pin.OUT)
leftStop = Pin(1,Pin.OUT)
rightTurn = Pin(2,Pin.OUT)
rightStop = Pin(4,Pin.OUT)
sensor = HCSR04(trigger_pin = 6, echo_pin = 7, echo_timeout_us = 10000)
#-------------------------------------------------

#-------------------------------------------------
#Turns all LEDs off
def LEDoff():
    leftTurn.off()
    rightStop.off()
    leftStop.off()
    rightTurn.off()
#-------------------------------------------------
```

```
#Movement Functions
#-------------------------------------------------
# Forward Movement
def forward():
    if forceStop == True: # if it is too close, will not be able to go
forward or turn
        stop()
    else:
        global dir
        lf.duty_u16(64500)
        rf.duty_u16(65536)
        lb.duty_u16(0)
        rb.duty_u16(0)
        dir = 0

# Reverse Movement
def backward():
    global dir
    global forceStop
    lb.duty_u16(64500)
    rb.duty_u16(65536)
    lf.duty_u16(0)
    rf.duty_u16(0)
    dir = 3
    forceStop = False #Clears the flag allowing for movement once it goes
back a little bit

# Turns Right
def TurnRight():
    if forceStop == True:
        stop()
    else:
        global dir
        lf.duty_u16(53000)
        rb.duty_u16(0)
        lb.duty_u16(0)
        rf.duty_u16(0)
        dir = 2

# Turns Left
def TurnLeft():
    if forceStop == True:
        stop()
    else:
        global dir
        lb.duty_u16(0)
        rf.duty_u16(53000)
        lf.duty_u16(0)
        rb.duty_u16(0)
        dir = 1
```

```
# Stops all Motors
def stop():
    global dir
    lf.duty_u16(0)
    rb.duty_u16(0)
    lb.duty_u16(0)
    rf.duty_u16(0)
    dir = 0
#-------------------------------------------------



#-------------------------------------------------
#Maps IR Codes to movement functions
def controller(code):
    if code == 0x18: #Up Button
        forward()
    elif code == 0x52: #Down Button
        backward()
    elif code == 0x08: #Left Button
        TurnLeft()
    elif code == 0x5A: #Right Button
        TurnRight()
    elif code == 0x1C: #OK Button
        stop()
#-------------------------------------------------


#-------------------------------------------------
#takes the data and puts it in the
def callback(data, addr, ctrl):
    controller(data)

#Checks for input in the background
ir = NEC_8(ir_pin,callback)
#-------------------------------------------------


#-------------------------------------------------
try:
    while True:
        if dir == 0:
            LEDoff()
        elif dir == 1: #Turns on Left Turn Signal
            leftTurn.on()
            utime.sleep(.5)
            leftTurn.off()
            utime.sleep(.5)
        elif dir == 2: #Turns on Right Turn Signal
            rightTurn.on()
            utime.sleep(.5)
            rightTurn.off()
            utime.sleep(.5)
        elif dir == 3: #Turns on Hazzards
```

```
            rightStop.on()
            leftStop.on()
            utime.sleep(.5)
            leftStop.off()
            rightStop.off()
            utime.sleep(.5)
        distance = sensor.distance_cm()
        if distance > 0 and distance < 10:
            stop()
            forceStop = True #Sets a flag that says that it is close to a
wall

        utime.sleep(0.1)
except KeyboardInterrupt:
    stop()
#------------------------------------------------
```

Conclusion:

Our project was a success and is fully remote control. The forward, reverse, left, and right work well. The prox sensor stops the robot from running into anything, the turn signals and the hazard lights work as intended. We had a slight issue with the robot not responding, we still don't know why it occurred, but it was fixed by unplugging and plugging it back in and then power cycling it. The PWM speed control didn't end up being used as we first intended, but we ended up using it to match the motor speeds so it would drive straight. We were happy with the outcome and learned a lot along the way.